Machine-Learned Algorithm Selection for Boolean Satisfiability: An Empirical Evaluation on a 2,299-Instance Corpus

A pre-registered comparison of a graph neural network selector, a SATzilla-style regression selector, and single-solver baselines

Ansh Jaiswal[1], Lakshit Verma[1], Aryendra Pandey[2], Priyanshu Sharma[2]

[1]Department of Electronics Engineering (VLSI), Manipal Institute of Technology, Manipal Academy of Higher Education

[2]Department of Electronics and Communication Engineering, Manipal Institute of Technology, Manipal Academy of Higher Education

Author	Registration no.	Programme	E-mail
Ansh Jaiswal	230959004	Electronics Engineering (VLSI)	ansh4.mitmpl2023@learner.manipal.edu
Lakshit Verma	230959210	Electronics Engineering (VLSI)	lakshit.mitmpl2023@learner.manipal.edu
Aryendra Pandey	230907498	Electronics and Communication Engineering	aryendra.mitmpl2023@learner.manipal.edu
Priyanshu Sharma	230907576	Electronics and Communication Engineering	priyanshu2.mitmpl2023@learner.manipal.edu

Report date: 30 August 2026

Software: `satsel`, Python 3.14, PySAT oracle wrappers, PyTorch Geometric

Protocol: pre-registered in `EVAL_PROTOCOL.md`; configuration hash `d442a795a2af`

ABSTRACT

Different Boolean satisfiability (SAT) solvers are fastest on different formulas, so choosing a solver per instance should in principle beat committing to one solver for every instance. This report tests whether that gain can be realised on a corpus of 2,299 formulas across eight families, using a portfolio of 7 conflict-driven clause-learning solvers. Three selectors were evaluated: a tripartite graph neural network operating on the formula structure, a SATzilla-style per-solver runtime regression, and a cost-sensitive classifier over 70 hand-designed features. All comparisons follow a protocol fixed before the models were fitted, using family-stratified paired bootstrap resampling with 10,000 resamples and Holm–Bonferroni correction across four planned tests.

On the held-out test set of 315 instances, no selector improved on the single best-average solver. Always running `cadical153` achieved a PAR-2 score of 6.333 s. The graph neural network selector scored 7.601 s (+20.0%) and the SATzilla-style baseline 7.595 s (+19.9%). The difference between the two selectors was 0.006 s (p = 0.9306), which does not distinguish them. The selector's deficit against the single best solver was statistically significant after correction (p = 0.0114).

The measured cause is the difficulty distribution of the corpus rather than the selection models. The median instance was decided in 14.3 ms, 61.9% of test instances had no solver meaningfully faster than the runner-up, and the single family carrying 77% of the total PAR-2 mass (*scheduling*) left only 1.4% headroom between the best single solver and the virtual best solver. Two component results were nonetheless obtained: the designed node features reduced validation PAR-2 by 1.654 s relative to a topology-only network, and increasing message-passing depth from one to four layers changed validation PAR-2 by less than 0.0002 s.

Keywords: algorithm selection; Boolean satisfiability; graph neural networks; portfolio solvers; PAR-2; empirical evaluation.

1. Introduction

A SAT solver takes a Boolean formula in conjunctive normal form and decides whether any assignment of true and false to its variables makes the whole formula true. Modern conflict-driven clause-learning (CDCL) solvers are mature and fast, but they are not uniformly fast: on the same formula one solver may finish in milliseconds while another exhausts a five-minute budget. Which solver wins varies with the structure of the formula, and no single solver dominates across problem types.

This observation motivates *algorithm selection* (Rice, 1976): rather than fixing one solver, inspect each incoming formula, predict which solver will be fastest on it, and run that one. The best-known realisation for SAT is SATzilla (Xu *et al.*, 2008), which computes a vector of cheap structural features from the formula, fits a runtime model per solver, and dispatches to whichever solver is predicted to be fastest. SATzilla won multiple SAT competition categories and remains the standard point of comparison.

The question this project set out to answer was fixed in advance and is stated in the pre-registered protocol:

> **Research question.** Does a machine-learned selector — classical or graph-neural — achieve lower PAR-2 and a higher solve rate than (i) the single best-average solver, (ii) uniform random selection, and (iii) a SATzilla-style regression baseline, on a completely held-out test set, with feature-extraction and orchestration overhead charged against the selector?

The answer obtained is no, for all three comparisons, and Section 5 sets out the measured reason. The evaluation itself, the corpus, and the component ablations are reported in full because they constrain what a subsequent attempt would need to change.

1.1 The evaluation metric

Results are reported in **PAR-2** (penalised average runtime, factor two), the standard scoring rule of the SAT competitions. For each instance the runtime of the chosen solver is recorded; if the solver does not finish within the time limit τ for that family, the instance is charged 2τ instead. The reported figure is the mean over all instances in the set. Lower is better, and the units are seconds. PAR-2 is used rather than plain mean runtime because plain mean runtime has no defined value when a solver times out, and rather than solve count alone because solve count ignores how long the solved instances took.

1.2 Systems compared

Nine systems appear in the results tables. They are defined once here and referred to by these names throughout.

Table 1. Systems evaluated. The three predictive systems are charged the cost of computing their own input features; the baselines are not, because they compute none.

System	Definition
best_average	Always run the solver with the lowest mean PAR-2 on the *training* split. Selected on training data only; this turned out to be `cadical153`. This is the baseline a selector must beat to be worth deploying.
gnn	The graph neural network selector: a tripartite variable/literal/clause graph passed through heterogeneous convolutions, trained with a runtime-sensitive expected-cost loss.
classical	The best of four cost-sensitive classifiers (random forest, gradient boosting, support vector machine, regret-based regressor) over 70 hand-designed features, chosen on the validation split.
satzilla	Our reimplementation of the SATzilla approach (Xu *et al.*, 2008): one runtime regression per solver, fitted on solved instances only, dispatching to the solver with the lowest predicted runtime. Fitted on the identical features and splits as the other selectors.
selector	The system that would actually be deployed: the selector chosen on validation (here the `gnn`), with the confidence-gated fallback applied. Because the fallback threshold was swept to $\theta = 0$, this row is identical to `gnn`.
fallback	The gated fallback path alone, with no model prediction used.
majority	Always run whichever solver was fastest most often on the training split. A zero-rule baseline requiring no features.
random	Uniform random choice among the 7 solvers, averaged over 10 seeded draws.
('**vbs** (virtual best solver)', 'An oracle that always picks the fastest solver for each instance. Not achievable in practice; it is the ceiling that bounds how much any selector could possibly gain.')	ceil

2. Background

2.1 Algorithm selection

Rice (1976) framed the general problem: given a space of problem instances, a set of algorithms, and a performance measure, find a mapping from instances to algorithms that optimises performance. For SAT the mapping is learned from data, which requires three ingredients: a set of instances broad enough to contain genuine variety, measured runtimes of every solver on every instance, and features of an instance that are cheap enough to compute that they do not consume the time the selection is meant to save.

The community benchmark for this line of work is ASlib (Bischl *et al.*, 2016), which packages algorithm-selection scenarios with fixed splits, and AutoFolio (Lindauer *et al.*, 2015), which searches over selector designs and hyperparameters automatically. Neither was used here; the corpus and selectors were built from scratch, which is a limitation discussed in Section 6.

2.2 SATzilla

SATzilla (Xu *et al.*, 2008) computes on the order of one hundred features per formula — clause and variable counts, clause-length statistics, graph properties of the variable–clause incidence structure, and probing features obtained by running a solver briefly and recording its internal counters — then fits an empirical hardness model per solver and selects the arg-min. It is reimplemented here rather than cited so that the comparison runs on identical instances, identical features, and identical splits. Two differences from the published system are noted for accuracy: the reimplementation uses random forests rather than ridge regression, and it does not include SATzilla's pre-solving schedule or its backup-solver logic.

2.3 Graph neural network selectors

Hand-designed features impose a ceiling: the selector can only distinguish formulas that the features distinguish. Graph neural networks avoid fixing the representation in advance by operating on the formula's graph structure directly. NeuroSAT (Selsam *et al.*, 2019) demonstrated that a message- passing network over a literal–clause graph can learn satisfiability- related structure from supervision alone. For selection specifically, the GraSS architecture (Li *et al.*, 2024)† combines a graph network with expert features rather than discarding them.

The network used here follows that design: separate node types for variables, literals and clauses over four relations, heterogeneous convolution, sinusoidal positional encodings on clause nodes, and a loss weighted by observed runtime so that mistakes on instances with large runtime spreads are penalised more than mistakes where every solver performs similarly. Giving each literal its own node is a correctness requirement, not a refinement: a variable–clause bipartite graph indexed by variable identity cannot distinguish $(x \vee y)$ from $(\neg x \vee \neg y)$.

3. Method

3.1 Solver portfolio

Seven CDCL solvers were used, all accessed through PySAT (Ignatiev *et al.*, 2018): `minisat22`, `glucose4`, `cadical153`, `maplesat`, `lingeling`, `maplecm` and `kissat404`. Every solver was run on every instance 3 times and the median of the solved runs recorded, to reduce the effect of machine noise on short-running instances.

A solver earns its place in a portfolio only if it is sometimes uniquely best. Table 2 reports the marginal contribution of each solver: the amount by which the virtual best solver's PAR-2 *worsens* if that solver is removed. A value near zero means the solver is redundant, because whenever it is best another solver is nearly as good.

Table 2. Marginal contribution of each solver to the virtual best solver, in seconds of PAR-2. Larger values indicate a solver that covers instances the others handle poorly.

Solver	Marginal PAR-2 increase if removed (s)	Status
maplecm	0.347	retained
glucose4	0.124	retained
kissat404	0.056	retained
cadical153	**0.054**	**best average solver**
minisat22	0.044	retained
maplesat	0.008	retained
lingeling	0.003	pruned as dominated

lingeling was pruned for failing the 3% strictly-best share threshold set in the configuration. The largest single contribution, maplecm at 0.347 s, is small relative to the corpus PAR-2 of 6.333 s, which is an early indication that the portfolio members are not strongly complementary on this corpus.

3.2 Instance corpus

The corpus contains 2,299 formulas across eight families. Seven families are in-distribution and were split into training, validation and test sets; the eighth, *ood_scheduling*, was held out entirely and never seen during training, so that generalisation to an unfamiliar problem type could be measured separately.

Roughly one fifth of the corpus consists of real benchmark instances downloaded from SATLIB (Hoos and Stützle, 2000) and the Global Benchmark Database. The remainder was generated by seeded generators for the families those archives do not cover. Random 3-SAT instances were generated at a clause-to-variable ratio near 4.26, the phase-transition point at which random 3-SAT instances are hardest (Mitchell, Selman and Levesque, 1992).

Splits are 70/15/15 and were made by a canonical hash of the formula content rather than by filename, so that duplicated or near-duplicated instances cannot appear on both sides of a split. 137 cluster duplicates were removed by this check, and the pipeline confirmed no residual leakage.

Table 3. Corpus composition and splits. The VBS solve rate is the fraction of instances that at least one solver in the portfolio could decide within the family time limit; it bounds the solve rate of every system.

Family	n	Train	Validation	Test	Time limit τ (s)	VBS solve rate
colouring	300	210	45	45	120	100.0%
crypto	100	70	15	15	300	100.0%
industrial	400	280	60	60	300	100.0%
ood_scheduling	199	0	0	0	300	96.5%
planning	200	140	30	30	120	100.0%
random3sat	400	280	60	60	60	100.0%
random3sat_large	400	280	60	60	60	100.0%
scheduling	300	210	45	45	180	95.0%
Total	**2,299**	**1,470**	**315**	**315**	—	—

ood_scheduling is the out-of-distribution family: it contributes no training, validation or test rows and is scored separately in Section 4.11.

3.3 Labelling and the tie rule

The label for an instance is the solver that solved it fastest. Two qualifications were fixed in advance, because raw arg-min labels are misleading when solvers finish at nearly the same time.

An instance is marked a **tie** if the second-fastest solver finished within 1.1× the time of the fastest, *or* if the fastest solver finished in under 20 ms. The first condition says the two solvers are not meaningfully different; the second says the measurement is dominated by process and measurement overhead rather than by search. An instance is marked **informative** for training only if the ratio between the slowest and fastest solver is at least 1.5, so that the model is trained on instances where the choice actually matters.

These two rates are the single most important diagnostic in this report. If most instances are ties, the labels carry little information and no model can recover a signal that is not there.

3.4 Features

70 features were computed per instance in three groups. *Syntactic* features (41) are counts and distributional statistics read directly off the clause list: variable and clause counts, the clause-to-variable ratio, clause-length and variable-occurrence distributions, the share of Horn clauses, and unit and binary clause counts. *Graph* features add properties of the variable–variable interaction graph, chiefly density and degree statistics. *Probing* features run a solver for a small fraction of the time budget and record its internal search counters.

Feature extraction is not free, and its cost is charged against the predictive systems in every PAR-2 figure reported here. Feature groups are entered atomically, because a half-computed group yields nothing usable, and group entry is decided against a size-based cost estimate.

3.5 Selection models

Four classical candidates were trained — random forest, gradient boosting, support vector machine, and a regret-weighted regressor — each with cost-sensitive weighting so that an expensive mistake counts for more than a cheap one. The candidate with the lowest validation PAR-2 was selected and carried forward as `classical`. The graph network was trained separately with a per-layer and per-feature-set ablation grid.

A confidence-gated fallback was also fitted: when the selector's confidence in its top choice falls below a threshold θ, the system defers to the fallback solver instead. θ was swept over a grid on the validation split.

3.6 Statistical protocol

The protocol was fixed before any model was fitted and is reproduced in EVAL_PROTOCOL.md . Four confirmatory comparisons were planned. Each was tested three ways, and the family-wise error rate across the four comparisons was controlled by the Holm–Bonferroni procedure (Holm, 1979) at $\alpha = 0.05$, so that the first comparison must clear $0.05/4 = 0.0125$.

- **Family-stratified paired bootstrap** (10,000 resamples; Efron, 1979). Instances are resampled with replacement within family, preserving the family mix, and the mean per-instance PAR-2 difference is recomputed each time. This yields a 95% confidence interval on the difference. Pairing matters: both systems are scored on the same instances, so instance difficulty cancels.
- **Wilcoxon signed-rank test** (Wilcoxon, 1945), a distribution-free test on the paired differences, restricted to instances both systems solved so that censored timeouts do not enter as ties.
- **McNemar's test** (McNemar, 1947) on the solve-rate difference, which asks only about instances where exactly one of the two systems succeeded.

The primary endpoint is the PAR-2 of the graph-network selector against the single best-average solver. A claim of improvement is made only if the bootstrap confidence interval excludes zero *and* the Holm-adjusted test is significant.

4. Results

4.1 Corpus and label characteristics

Table 4 summarises the labels on the test split. Three numbers determine how much room any selector has to operate.

Table 4. Label characteristics of the test split.

Quantity	Value	Interpretation
Test instances	315	Size of the held-out evaluation set.
VBS solve rate	98.73%	Fraction decided by at least one solver; the ceiling on every system's solve rate.
Most frequent best solver	kissat404 (30.9%)	A constant predictor attains this classification accuracy without any features.
Tie rate	61.9%	Instances with no solver meaningfully faster than the runner-up. These contribute no learnable signal.
Informative-label rate	53.7%	Instances where the slowest solver takes at least 1.5× the fastest.
Median time of the fastest solver	14.3 ms	Half the test instances are decided faster than this. The configured difficulty threshold for a usable corpus is 50 ms.

The pipeline emitted an automated warning on both highlighted rows. A tie rate above 50% and a median best-solve time below 50 ms are the two conditions under which the labelling scheme itself declares the corpus insufficiently discriminating.

Two of the six warnings the pipeline raised concern this table directly, and they anticipate the main result: on a corpus where the median instance is decided in 14.3 ms and 61.9% of instances have no clear winner, the quantity a selector is trying to predict is mostly noise. Section 5 quantifies how much room this leaves.

Table 5 shows which solver was fastest, and how often. No solver is fastest on more than a third of instances, which is the condition that makes selection worth attempting in the first place.

Table 5. Distribution of the fastest solver over the test split.

Fastest solver	Instances	Share
kissat404	**96**	**30.9%**
minisat22	88	28.3%
cadical153	55	17.7%
glucose4	48	15.4%
maplecm	10	3.2%
maplesat	10	3.2%
lingeling	4	1.3%

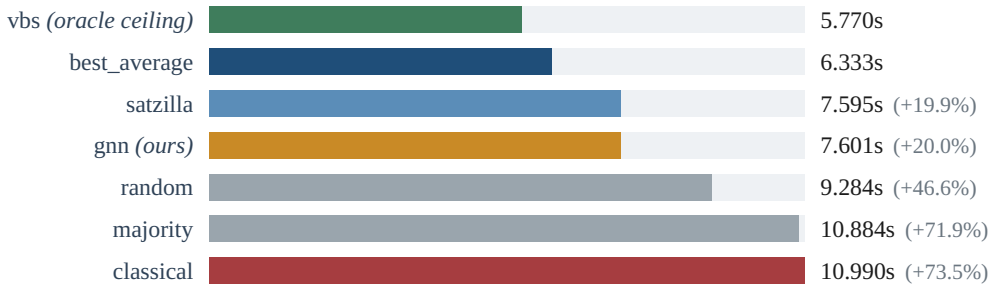
Counts sum to the instances the virtual best solver decided; the remainder timed out under every solver.

4.2 Main result

PRIMARY ENDPOINT

The graph-network selector scored **7.601 s** PAR-2 on the test set against **6.333 s** for always running `cadical153`. The selector is **20.0% worse** than the single best solver, and this deficit is significant after Holm–Bonferroni correction ($p = 0.0114$). The pre-registered claim of improvement is not supported.

Figure 1. Test-set PAR-2 by system, in seconds. Lower is better. The green bar is the unreachable oracle; the dark blue bar is the baseline any deployable selector must beat.



The gap between the green and dark blue bars is the entire opportunity available to any selector on this corpus: 0.562 s, or 8.88% of the baseline. Every selector shown lands above the baseline rather than inside that gap.

Table 6. All systems on the test set. The percentage column is relative to `best_average`; negative would indicate an improvement.

System	PAR-2 (s)	vs. baseline	Solve rate	n
best_average	**6.333**	**—**	**98.73%**	**315**
fallback	6.333	+0.00%	98.73%	315
vbs (oracle)	*5.770*	*-8.88%*	*98.73%*	*315*
satzilla	7.595	+19.93%	98.10%	315
selector	7.601	+20.03%	98.10%	315
gnn	7.601	+20.03%	98.10%	315
random	9.284	+46.60%	97.46%	315
majority	10.884	+71.86%	97.46%	315
classical	10.990	+73.54%	97.46%	315

`selector` and `gnn` are identical because the fallback threshold was swept to $\theta = 0$ (Section 4.10), so no prediction was ever overridden. `fallback` equals `best_average` for the same reason. The random baseline is the mean of 10 seeded draws, standard deviation 1.104 s.

4.3 Confirmatory statistical tests

Table 7 gives the four planned comparisons. A positive mean difference means the first system is *slower*, since the metric is a cost. The confidence interval is the family-stratified paired bootstrap over 10,000 resamples; the Holm column states whether the comparison survives correction for having made four tests.

Table 7. The four pre-registered confirmatory comparisons.

Comparison	Mean difference (s)	95% CI	Bootstrap *p*	Wilcoxon *p*	McNemar *p*	Holm reject
Selector vs. best single solver *(primary)*	+1.2686	[+0.059, +3.378]	0.0114	< 0.0001	1.000	**yes**
Selector vs. random selection	-1.6827	[-2.977, +0.013]	0.0526	< 0.0001	1.000	no
Selector vs. SATzilla-style regression	+0.0064	[-0.097, +0.119]	0.9306	0.4567	1.000	no
Classical classifier vs. best single solver	+4.6569	[+1.266, +9.064]	0.0000	< 0.0001	0.250	**yes**

Wilcoxon *n* is 309 for the first three comparisons and 307 for the fourth, restricted to instances both systems solved. McNemar's test is uninformative throughout because the systems disagree on solve outcome for at most four instances.

Reading the four rows in turn:

- **Against the best single solver**, the selector is 1.269 s slower per instance. The confidence interval [0.059, 3.378] excludes zero and the result survives correction. This is the primary endpoint and it goes against the hypothesis.
- **Against random selection**, the selector is 1.683 s faster, but the interval [-2.977, 0.013] includes zero and the bootstrap *p* of 0.0526 does not clear the corrected threshold. The selector cannot be shown to beat random choice at the pre-registered level. The Wilcoxon test does reject ($p < 0.0001$), which means the selector wins on more instances than it loses; the mean is nonetheless not separable from zero because a small number of instances with large PAR-2 dominate it.
- **Against the SATzilla-style baseline**, the difference is 0.0064 s with interval [-0.097, 0.119] and $p = 0.9306$. The two selectors are indistinguishable on this corpus. Neither is better than the other, and both are worse than running one solver.
- **The classical classifier** is 4.657 s worse than the baseline, significant at $p < 0.0001$. It is also worse than random selection. Section 5.2 identifies the cause.

4.4 Where the difficulty actually sits

A mean over the whole test set hides how unevenly the work is distributed. Figure 2 decomposes the total PAR-2 by family: each family's share is its mean PAR-2 multiplied by its instance count, as a percentage of the total.

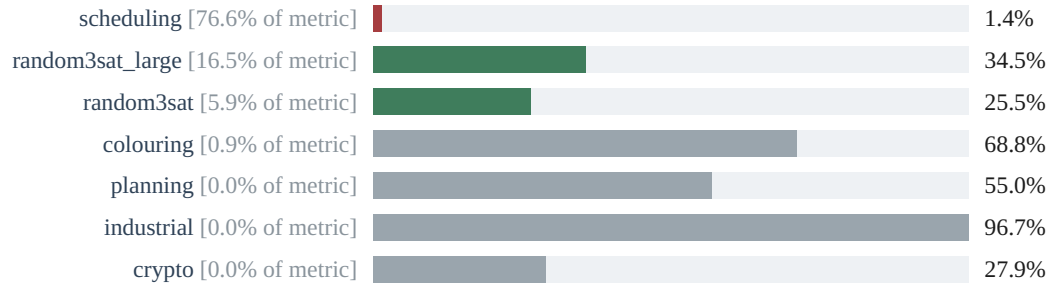
Figure 2. Share of total test-set PAR-2 contributed by each family. The metric is almost entirely determined by one family.



Four of the seven families together account for under 1% of the total. On those families every solver finishes in single-digit milliseconds, so nothing a selector does there can move the overall score.

The consequence is that the headline number is, in effect, a measurement on the *scheduling* family alone. Figure 3 pairs each family's share of the metric with the room available on it — the percentage gap between always running the best single solver and the oracle. A family matters to the result only if both bars are substantial.

Figure 3. Oracle headroom by family: the percentage by which the virtual best solver improves on the single best solver within that family. This is the maximum any selector could gain there.



The two bars are in tension. *scheduling* carries 77% of the metric but offers only 1.4% headroom — `cadical153` is already within a rounding error of optimal there. *industrial* offers 96.7% headroom but carries 0.015% of the metric, so winning it completely would change the overall score by nothing measurable. Only *random3sat_large* combines meaningful weight (16.5%) with meaningful room (34.5%).

> **The binding constraint.** Across the whole test set the oracle improves on the single best solver by only 0.562 s, which is 8.88% of the baseline. Feature extraction alone costs the predictive systems a mean of 0.0960 s per instance. A selector must therefore find most of an 8.9% opportunity merely to break even against a baseline that computes nothing at all.

4.5 Per-family results

Table 8 gives every system's PAR-2 within every family. This is where the aggregate result becomes legible, and where the two cases in which a selector did win appear.

Table 8. Per-family PAR-2 in seconds. A filled circle marks the best achievable system in that family, excluding the oracle.

Family	n	best_average	majority	gnn / selector	classical	satzilla	vbs (oracle)	random
['scheduling', '45', '33.973 ●', '64.978', '40.915', '65.122', '40.668', '33.511', '38.617']								
['random3sat_large', '60', '5.493 ●', '5.940', '6.549', '6.085', '6.860', '3.599', '13.695']								
['random3sat', '60', '1.969 ●', '2.050', '2.065', '2.164', '2.011', '1.467', '4.444']								
['colouring', '45', '0.384 ●', '0.546', '0.615', '0.615', '0.476', '0.120', '2.103']								
['planning', '30', '0.025', '0.015 ●', '0.228', '0.228', '0.228', '0.011', '0.114']								
['industrial', '60', '0.005', '0.000 ●', '0.030', '0.030', '0.030', '0.000', '0.005']								
['crypto', '15', '0.000 ●', '0.000', '0.003', '0.003', '0.003', '0.000', '0.000']								

`gnn` and `selector` are the same system. Solve rates are given in the accompanying CSV (`03_per_family_par2.csv`); they are 100% in every family except *scheduling* and *random3sat_large*.

Two observations follow from Table 8.

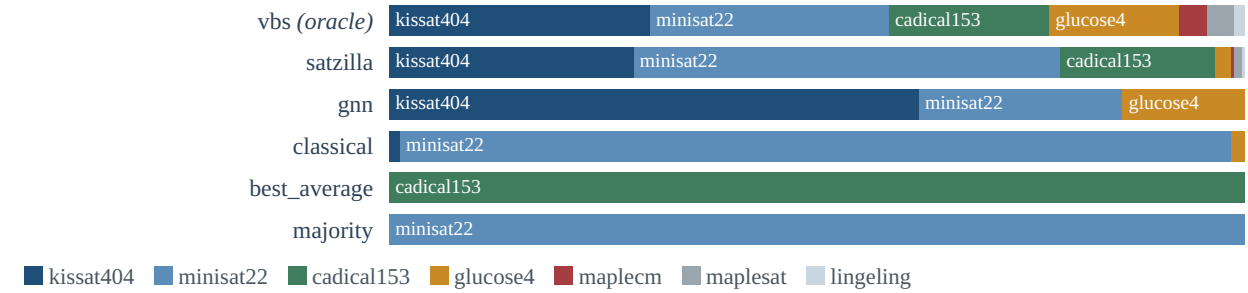
First, the selector was beaten by `best_average` in every single family. There is no family in which choosing per instance paid off against committing to `cadical153`.

Second, in the comparison against the SATzilla-style baseline the outcome is split. Our selector was better on *random3sat_large* (6.549 s against 6.860 s, an improvement of 4.5%), which is the family with the most headroom among those carrying real weight. The SATzilla-style baseline was better on *colouring*, *random3sat* and *scheduling*. Because these per-family differences were not part of the four pre-registered comparisons, no significance can be claimed for them individually; they are reported as observations, not results.

4.6 Which solvers each system chose

Comparing a system's choice distribution against the oracle's shows whether the system is discriminating at all, or has collapsed onto a constant.

Figure 4. Solver choice distribution by system on the test set, compared with the oracle's distribution at the top.



The oracle spreads across all seven solvers. satzilla spreads across seven and gnn across three. classical puts 97% of its mass on one solver, which is the collapse the pipeline flagged.

4.7 Selection accuracy

PAR-2 measures the consequence of a choice; classification accuracy measures the choice itself. Accuracy is computed only on the 116 non-tie test instances, since on a tie there is no correct answer to get right. Cost-weighted accuracy credits a near-miss more than a catastrophic one: it averages the reciprocal of the runtime ratio between the chosen solver and the best solver, so 1.0 means always optimal.

Table 9. Selection accuracy on non-tie instances. The majority-class reference is the accuracy obtained by always naming kissat404.

System	Accuracy	Cost-weighted accuracy	n scored	colouring	planning	random3sat	random3sat_large	scheduling
gnn / selector	25.0%	0.711	116	21.4%	100.0%	11.6%	35.4%	12.5%
classical	35.3%	0.664	116	21.4%	100.0%	53.5%	22.9%	12.5%
satzilla	31.9%	0.765	116	7.1%	100.0%	41.9%	22.9%	50.0%
majority-class reference	30.9%	—	—	—	—	—	—	—

crypto and *industrial* are omitted: every instance in them is a tie, so no instance is scorable. Per-family accuracies are computed on small counts and are indicative only.

Three points are worth drawing out. The classical classifier attains the highest raw accuracy (35.3%) yet the worst PAR-2 of any system, because its errors are expensive ones; its cost-weighted accuracy (0.664) is the lowest in the table. The SATzilla-style baseline has the best cost-weighted accuracy (0.765), consistent with its design objective being predicted runtime rather than predicted rank. And every system except the classical classifier sits at or below the 30.9% obtained by naming one solver for everything, which is the clearest statement that little was learned.

4.8 Graph network ablations

The network was trained nine times, over three node-feature settings and three depths, on identical splits and seeds. *full* uses the designed per-node-type features; *structural* keeps only degree information; *none* replaces every node feature with a constant, leaving the network nothing but topology and message passing.

Table 10. Validation PAR-2 in seconds across the ablation grid. Lower is better.

Node features	1 layer	2 layers	4 layers	Final training loss	Distinct solvers chosen
full (designed features)	**2.3031**	**2.3029**	**2.3029**	**1.166**	**3 of 7**
structural (degree only)	2.3465	2.3465	2.3465	1.272	2 of 7
none (topology only)	3.9565	3.9565	3.9565	3.913	1 of 7

Each cell is a separate training run. The validation set here contains 312 of the 315 instances; three exceeded the graph size cap of 500,000 nodes and were routed to the classical path.

Figure 5. Effect of node features on validation PAR-2, holding architecture and seeds fixed (objective O6).



The three variants differ only in node features. The gap is therefore attributable to the features alone.

> **Result O6.** Replacing the designed node features with constants worsens validation PAR-2 by 1.654 s (3.956 s against 2.303 s), and the topology-only network collapses to naming `kissat404` on all 312 validation instances. On this corpus, message passing over the formula graph does not recover what the hand-designed features supply.

> **Result on depth.** Across one, two and four message-passing layers, validation PAR-2 varies by less than 0.0002 s within every feature setting, and the choice histograms differ by at most one instance. Additional message passing changed nothing measurable. Together with the previous result, this indicates the network is acting on its node features rather than on the graph structure those nodes sit in.

4.9 Feature-cost ablation

Features cost time to compute, and that time is charged against the selector. Table 11 refits a fixed random forest on cumulative feature groups to ask whether the expensive groups pay for themselves.

Table 11. Cumulative feature groups, each scored on validation with an identical model (100 trees, depth 8, balanced classes).

Feature group	Features	Validation PAR-2 (s)	Accuracy	Mean extraction cost (s)
syntactic	41	5.938	32%	0.0352
syntactic + graph	**60**	**5.841**	**37%**	**0.0352**
all	70	6.368	34%	0.0960

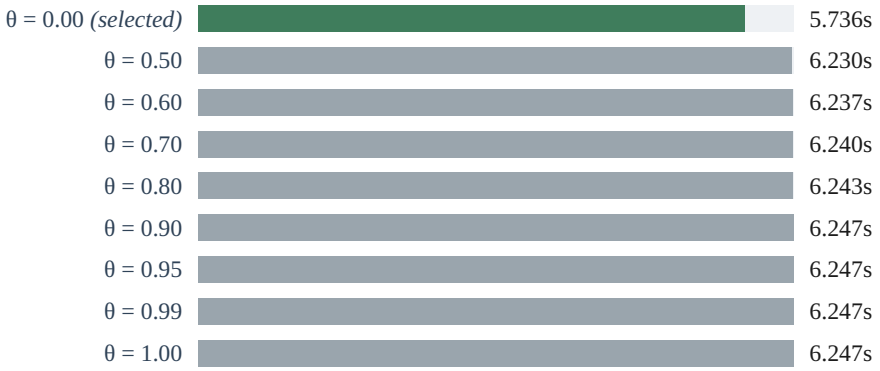
The `all` group adds probing features, which require running a solver briefly on the instance. Extraction cost for the partial groups is approximated as total feature cost minus probe runtime.

Adding the probing features made the system *worse* on both axes: they raised validation PAR-2 from 5.841 s to 6.368 s while nearly tripling extraction cost from 0.0352 s to 0.0960 s. On a corpus where the median instance is solved in 14.3 ms, spending 96 ms probing it is not recoverable. The Pareto-preferred configuration is *syntactic + graph*, which is both cheaper and better.

4.10 The confidence-gated fallback

The fallback deferring to a default solver when the model is unsure was swept over a grid on validation. Every non-zero threshold made things worse, and the curve is monotonic.

Figure 6. Validation PAR-2 as a function of the fallback confidence threshold θ. θ = 0 disables the fallback entirely.



The selected value θ = 0 means the mechanism was switched off. Deferring to the default solver whenever the model was unsure never helped, at any threshold tested. The same outcome was obtained in an earlier pilot run on a different corpus.

4.11 Out-of-distribution family

The *ood_scheduling* family (199 instances) was excluded from training entirely. It tests whether a selector fitted on seven families transfers to an eighth it has never seen.

Table 12. Performance on the held-out out-of-distribution family.

System	PAR-2 (s)	vs. best_average	Solve rate	Choices made
vbs (oracle)	*25.983*	*-29.2%*	*96.5%*	—
best_average	**36.723**	**—**	**94.5%**	**cadical153 100%**
selector	36.338	-1.0%	94.5%	glucose4 100%
classical	68.747	+87.2%	89.4%	minisat22 93%, glucose4 7%
satzilla	51.998	+41.6%	91.5%	minisat22 74%, cadical153 15%, glucose4 7%

The virtual best solver decides 96.5% of this family, which is well above the 30% floor below which the comparison would be vacuous. The leave-one-family-out sweep was not run; it requires retraining once per family and was outside the compute budget.

This is the only setting in which the selector finished ahead of the baseline, at 36.338 s against 36.723 s, a margin of 1.0%. It also finished well ahead of the SATzilla-style baseline (51.998 s, a margin of 30%).

The result should not be read as evidence of successful transfer. The selector chose glucose4 on all 199 instances, so it made no per-instance distinctions at all; the outcome reflects a single constant choice that happened to suit this family, on a margin of 0.385 s. What the comparison does show is a genuine difference in failure behaviour: the SATzilla-style baseline and the classical classifier both transferred badly, placing most of their mass on minisat22 and losing 42% and 87% respectively against the baseline, along with three to five percentage points of solve rate.

5. Discussion

5.1 Why no selector beat the single best solver

The evidence points to the composition of the corpus rather than to the selection models. Four measurements support this.

There was almost nothing to win. The oracle beats the single best solver by 8.88% overall. This is the total budget available to a perfect selector with free features. Published gains for portfolio selectors are reported against benchmark sets where the oracle gap is several times larger.

The gap that exists is in the wrong place. The *scheduling* family carries 77% of the metric and offers 1.4% headroom, because `cadical153` is already close to optimal on it. The families with large headroom (*industrial*, 96.7%; *colouring*, 68.8%) contribute under 1% of the metric between them. A selector could be perfect on those families and the overall number would not move.

The labels carry little signal. 61.9% of test instances are ties under the pre-registered rule, and the median instance is decided in 14.3 ms. At that scale, measured differences between solvers are comparable to process start-up and measurement noise. The pipeline's own data-quality check flags this: its threshold for a corpus capable of separating solvers is a median best-solve time of at least 50 ms, and the corpus is a factor of 3.5 below it.

Overhead is charged against the selector but not the baseline. Feature extraction costs 96 ms per instance on average, against a median instance runtime of 14.3 ms. On the majority of instances the selector spends longer deciding than the solver spends solving. This is the correct accounting — a deployed system pays it — but it means the selector begins each easy instance already behind.

Taken together: the accessible gain is 8.9%, most of it sits in families that contribute nothing to the metric, and the entry cost of participating is a fixed overhead that on most instances exceeds the runtime being optimised. Both selectors landing about 20% above the baseline is consistent with this account, as is the fact that they landed there together ($p = 0.93$) despite entirely different model families, feature treatments and training objectives. Two unrelated methods failing identically is more readily explained by the data than by a shared modelling error.

5.2 The classical classifier

The classical path failed in a way worth recording separately, because the mechanism is a model-selection error rather than a corpus property.

Table 13. Classical model selection on the validation split. The candidate with the best validation PAR-2 was carried forward.

Candidate	Validation PAR-2 (s)	Validation accuracy	Inner cross-validation score	Outcome
random forest	5.884	39.3%	-2.736	not selected
gradient boosting	6.036	38.6%	-3.025	not selected
support vector machine	5.736	35.0%	-4.351	**selected**
regret regressor	6.298	37.1%	-2.552	not selected

The inner cross-validation score is a negated PAR-2, so a value closer to zero is better. The selected candidate has the best single-split validation score and the worst cross-validated score of the four.

The support vector machine was chosen because it had the lowest validation PAR-2, 5.736 s, ahead of the random forest at 5.884 s. On the inner cross-validation, however, it ranked last of the four (-4.351 against -2.552 for the best). On the test split it placed 306 of 315 instances (97%) on `minisat22` and scored 10.990 s, worse than random selection.

The selection rule was therefore reading noise. A single validation split of 315 instances, on a corpus where most instances are ties and a few carry nearly all the PAR-2 mass, does not have the resolution to separate candidates that differ by 0.15 s. The cross-validated score, which had the resolution, disagreed and was not used. Selecting on cross-validation rather than on a single split is a concrete correction, independent of any change to the corpus.

5.3 Relation to published results

SATzilla's reported advantage was obtained on SAT competition instances, which are curated to be hard: instances solvable in milliseconds do not survive into a competition set. The present corpus is four fifths generated, and the generators were calibrated for solvability rather than for a target difficulty band. The consequence is visible in Table 4.

Our reimplementation of SATzilla is also not the full published system: it omits the pre-solving schedule and the backup-solver logic, and it substitutes random forests for ridge regression. It should be read as a like-for-like regression baseline on our data, not as a measurement of SATzilla itself. The statement supported by Table 7 is that a runtime-regression selector and a graph-network selector perform the same on this corpus, and that neither beats a single solver here.

6. Limitations

1. **Corpus difficulty.** The median instance is decided in 14.3 ms against a design threshold of 50 ms, and 61.9% of test instances are ties. This is the dominant limitation and it bounds every conclusion about selector performance. The corpus does not establish that these selectors would fail on a harder set.
2. **Generated majority.** Only about 20% of instances are real benchmark formulas; the rest come from seeded generators. Generated instances within a family are more homogeneous than competition instances, which reduces the variety a selector could exploit.
3. **Family imbalance in the metric.** One family supplies 77% of the PAR-2 mass. The aggregate is close to a single-family measurement, and a different family mix would produce a different headline number without any change to the models.
4. **Per-family comparisons are not confirmatory.** Only the four comparisons in Table 7 were pre-registered. The per-family differences in Table 8, including the *random3sat_large* result, carry no significance claim.
5. **Incomplete protocol coverage.** The leave-one-family-out sweep and the robustness appendix were not run, both for compute reasons. They are recorded as not run rather than omitted.
6. **Reimplemented baseline.** As set out in Section 5.3, the SATzilla comparison is against a reimplementation, not the published system.
7. **Single hardware configuration.** All runtimes come from one machine. Solver rankings on short-running instances are sensitive to hardware, and the three-run median mitigates but does not eliminate this.

Automated data-quality warnings issued by the pipeline. Reproduced verbatim, in the order emitted.

- [dataset_summary] median best time 26.2ms < 0.05s: corpus too easy to differentiate solvers; increase generator sizes
- [dataset_summary] only 20% of the corpus is real benchmark instances; the rest is generated

- [model_diagnostics] calibration of svm worsened val PAR-2 (9.6111 vs 5.7361); kept uncalibrated model
- tie rate 0.62 > 0.5: most formulas have no clear best solver (solve times below the tie floor); labels carry little signal
- median best solve time 14 ms: formulas are too easy for solver differences to exceed overhead noise
- system 'classical' picks 'minisat22' on 97% of test formulas (classifier collapse)

7. Conclusions

On a held-out test set of 315 instances drawn from a 2,299-formula corpus, and following a protocol fixed before the models were fitted:

1. Neither the graph-network selector (7.601 s) nor the SATzilla-style regression selector (7.595 s) improved on always running `cadical153` (6.333 s). The graph network's deficit is significant after correction for multiple comparisons ($p = 0.0114$).
2. The two selectors could not be distinguished from one another (difference 0.0064 s, $p = 0.93$).
3. The selector could not be shown to beat uniform random selection at the pre-registered significance level ($p = 0.0526$).
4. The measured cause lies in the corpus. Total oracle headroom is 8.88%; 77% of the metric sits in one family that offers 1.4% headroom; 61.9% of instances are ties; and the median instance is decided in 14.3 ms against a design threshold of 50 ms.
5. Designed node features contributed measurably to the graph network (1.654 s of validation PAR-2), while message-passing depth from one to four layers did not (under 0.0002 s).
6. Probing features did not pay for their extraction cost: removing them improved validation PAR-2 from 6.368 s to 5.841 s at roughly one third of the cost.
7. Confidence-gated fallback did not help at any threshold tested; the swept optimum switched the mechanism off.

8. Further work

The results identify specific changes, in the order they would matter.

1. **Retarget the corpus to a difficulty band.** The generators should be calibrated so that the median fastest solve time falls between roughly 1 and 10 seconds and the tie rate falls below 30%, rather than being calibrated for solvability alone. A short difficulty sweep can confirm the band before a full profiling run is committed. This is the change on which every other change depends.
2. **Rebalance the family mix.** *crypto*, *industrial* and *planning* contribute 0.05% of the metric between them at their current sizes. Effort should move to *random3sat_large*, which combines meaningful weight with 34% headroom, and to a harder *scheduling* variant where the best single solver is not already near-optimal.
3. **Select classical models on cross-validation.** Section 5.2 shows a single 315-instance validation split choosing the candidate that cross-validation ranked worst. This correction is independent of the corpus and should be made regardless.
4. **Drop the probing feature group** unless the corpus is retargeted to instances long enough to amortise it (Section 4.9).

5. **Adopt an established benchmark for comparison.** Running the same selectors on an ASlib SAT scenario (Bischl *et al.*, 2016) would separate corpus effects from model effects directly, and would give a reference point that does not depend on our generators.
6. **Compare against a configured selector.** AutoFolio (Lindauer *et al.*, 2015) searches selector designs automatically and would establish whether a tuned classical selector clears the bar our hand-built ones did not. Instance-based selection (Nikolić, Marić and Janićić, 2009) is a cheap additional comparison, and its local neighbourhood approach may behave differently from the global regressors used here.
7. **Complete the protocol.** Run the leave-one-family-out sweep and the robustness appendix, both deferred here for compute reasons.

References

Bischl, B., Kerschke, P., Kotthoff, L., Lindauer, M., Malitsky, Y., Fréchette, A., Hoos, H.H., Hutter, F., Leyton-Brown, K., Tierney, K. and Vanschoren, J. (2016) 'ASlib: A benchmark library for algorithm selection', *Artificial Intelligence*, 237, pp. 41–58.

Efron, B. (1979) 'Bootstrap methods: another look at the jackknife', *The Annals of Statistics*, 7(1), pp. 1–26.

Holm, S. (1979) 'A simple sequentially rejective multiple test procedure', *Scandinavian Journal of Statistics*, 6(2), pp. 65–70.

Hoos, H.H. and Stützle, T. (2000) 'SATLIB: An online resource for research on SAT', in Gent, I.P., van Maaren, H. and Walsh, T. (eds.) *SAT 2000*. Amsterdam: IOS Press, pp. 283–292.

Ignatiev, A., Morgado, A. and Marques-Silva, J. (2018) 'PySAT: A Python toolkit for prototyping with SAT oracles', in *Theory and Applications of Satisfiability Testing – SAT 2018*. Cham: Springer, pp. 428–437.

Li, Z., Guo, J., Jiang, Y. and Si, X. (2024)† 'GraSS: Combining graph neural networks with expert knowledge for SAT solver selection', in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. New York: ACM.

Lindauer, M., Hoos, H.H., Hutter, F. and Schaub, T. (2015) 'AutoFolio: An automatically configured algorithm selector', *Journal of Artificial Intelligence Research*, 53, pp. 745–778.

McNemar, Q. (1947) 'Note on the sampling error of the difference between correlated proportions or percentages', *Psychometrika*, 12(2), pp. 153–157.

Mitchell, D., Selman, B. and Levesque, H. (1992) 'Hard and easy distributions of SAT problems', in *Proceedings of the Tenth National Conference on Artificial Intelligence (AAAI-92)*. San Jose, CA, pp. 459–465.

Nikolić, M., Marić, F. and Janićić, P. (2009) 'Instance-based selection of policies for SAT solvers', in *Theory and Applications of Satisfiability Testing – SAT 2009*. Berlin: Springer, pp. 326–340.

Rice, J.R. (1976) 'The algorithm selection problem', *Advances in Computers*, 15, pp. 65–118.

Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L. and Dill, D.L. (2019) 'Learning a SAT solver from single-bit supervision', in *International Conference on Learning Representations (ICLR 2019)*.

Wilcoxon, F. (1945) 'Individual comparisons by ranking methods', *Biometrics Bulletin*, 1(6), pp. 80–83.

Xu, L., Hutter, F., Hoos, H.H. and Leyton-Brown, K. (2008) 'SATzilla: Portfolio-based algorithm selection for SAT', *Journal of Artificial Intelligence Research*, 32, pp. 565–606.

† This entry is carried over from the project synopsis. Its bibliographic details should be checked against the original source before submission.

Appendix A. Experimental configuration

Table A1. Configuration values governing the experiment. The full file is included in the data package as `default.yaml`.

Parameter	Value	Meaning
Portfolio	7 CDCL solvers	minisat22, glucose4, cadical153, maplesat, lingeling, maplecm, kissat404
Runs per instance–solver pair	3	Median of solved runs recorded
Family time limits τ	60–300 s	Per family; see Table 3
Screening budget	30 s	First-pass run budget during profiling
Tie margin	1.1×	Second-fastest within this ratio counts as a tie
Tie floor	20 ms	Fastest solve below this counts as a tie
Informative-label spread	1.5×	Minimum slowest-to-fastest ratio for a training row
Cost clip	10×	Upper clamp on the cost-weighting ratio
Split ratio	70 / 15 / 15	By canonical content hash, cluster-safe
Graph size cap	500,000 nodes	Instances above this are routed to the classical path
Bootstrap resamples	10,000	Family-stratified, paired; uncapped at this test-set size
Multiple-comparison correction	Holm–Bonferroni	α = 0.05 across 4 planned comparisons
Seeds	split 42, generators 2026, baseline 2026	Model training used seeds 100, 200, 300, 400, 500
Configuration hash	`d442a795a2af`	SHA-256 prefix; schema 1.0, feature schema 1.1

Appendix B. Accompanying data files

Every figure in this report is reproducible from the files in the `data/` folder. CSVs are UTF-8 with a header row.

Table B1. Contents of the data package.

File	Contents
`01_systems_test.csv`	Table 6 — every system's test PAR-2, solve rate and relative difference
`02_planned_comparisons.csv`	Table 7 — the four confirmatory tests with CIs and all three *p*-values
`03_per_family_par2.csv`	Table 8 in long form, including solve rates
`04_par2_mass_and_headroom.csv`	Figures 2 and 3 — PAR-2 mass and oracle headroom by family
`05_label_distribution.csv`, `05b_label_summary.csv`	Tables 4 and 5
`06_choice_histograms.csv`	Figure 4 — solver choices per system
`07_classification_accuracy.csv`	Table 9, including per-family accuracy
`08_feature_ablation.csv`	Table 11
`09_gnn_ablation.csv`	Tables 10 and Figure 5 — the full 3×3 grid
`10_ood_results.csv`	Table 12
`11_corpus_composition.csv`	Table 3
`12_portfolio_marginal.csv`	Table 2
`13_model_selection_val.csv`	Table 13
`14_theta_curve.csv`	Figure 6
`15_data_quality_warnings.csv`	The six automated warnings
`results.json`, `ood.json`	Raw evaluation output, unmodified
`dataset_summary.json`	Corpus composition and leakage check
`model_diagnostics.json`, `theta.json`, `portfolio.json`	Model selection, threshold sweep, portfolio analysis
`report.md`	The pipeline's own generated report
`EVAL_PROTOCOL.md`	The pre-registered protocol and its deviations log
`default.yaml`	Complete experimental configuration

Appendix C. Reproducibility

The pipeline runs in six stages, each writing to `artifacts/`: corpus construction, feature extraction, solver profiling, dataset assembly with splits, model training, and evaluation. The evaluation stage reads only the test split and is the only stage that does so.

Random seeds are fixed in configuration for splitting (42), instance generation (2026), the random baseline (2026), and model training (100, 200, 300, 400, 500). The configuration hash `d442a795a2af` covers the values in Appendix A; a changed hash indicates results are not comparable with those reported here.

Profiling ran 7 solvers × 2,299 instances × 3 repetitions. Model training used one NVIDIA RTX 4050 Laptop GPU for the graph network and CPU for the classical models. Deviations from the pre-registered protocol, including the generator recalibration carried out after the first pilot run, are logged in Section 8 of `EVAL_PROTOCOL.md`.